Day – 19 Introduction to NLP

Natural Language Processing (NLP) helps computers understand, interpret and produce human language. It studies language as data and develops a model that can analyse linguistic structure, meaning and context in both written and spoken communication.

Simple Example of NLP: “Ravi is happy with the new phone.”

An NLP system can:

- Detect Ravi as a person
- Identify phone as an object
- Recognize sentiment as positive
- Understand topic as product review

How Natural Language Processing Work:

1. Text or Speech Input

- The system takes written language like sentences or documents which is called text acquisition.
- When the input is audio, it is first converted into text using Speech Recognition.

2. Pre-processing

The text is cleaned and prepared. It can include:

- Cleaning unwanted characters or symbols from text is done using text normalization.
- Breaking sentences into smaller units so they can be processed easily.
- Changing all words into the same case for uniform processing is known as case folding.
- Eliminating frequent or common words like is, the, and to focus on meaningful terms.
- Converting words like running to run (base form) to reduce computational power.

3. Language Analysis

The system studies structure and meaning:

- Identifying nouns, verbs, and other parts of speech in a sentence is done.
- Finding how words connect to each other in a sentence.
- Determining the actual meaning of a word based on surrounding text.
- Detecting entities like person names, locations, or dates.
- Identifying whether text expresses positive, negative or neutral emotion.

4. Text Representation and Embedding Techniques

Since machines process numbers, this stage converts text into numerical vectors.

- **Text representation:** In this step, text is converted into numbers using statistical features or vector representations so machines can process it.
- **Traditional representations:** Earlier methods represent text using word counts and importance scores.
- **Word embeddings:** Modern methods represent words as dense vectors capturing similarity and meaning.
- **Contextual embeddings:** Advanced models generate word meanings based on the surrounding sentence.

5. Model Training

Once text is numeric, models are trained to learn patterns and perform NLP tasks.

- After text is converted into vectors, algorithms learn patterns from data to perform tasks like classification or translation.
- Earlier NLP systems relied on statistical algorithms that learn from manually prepared features.
- Modern NLP uses neural networks that automatically learn language structure from large data.
- Large language models trained on massive datasets can be reused and fine-tuned for tasks.

6. Output Generation

The system produces results such as:

- Text reply
- Voice response
- Translation
- Summary
- Prediction

Common NLP Tasks

- **Text classification:** Assigning predefined labels to text like spam or topic categories.
- **Sentiment analysis:** Detecting whether text expresses positive, negative or neutral emotion.
- **Machine translation:** Automatically converting text from one language to another.
- **Named Entity Recognition:** Identifying names of people, places, dates, etc in text.

- **Text summarization:** Generating a shorter version of a document while keeping key meanings.
- **Question answering systems:** Systems that read text and return exact answers to queries.

Real-Life Applications

- Voice assistants like Alexa, Google Assistant, etc
- Chatbots in customer support
- Email spam filtering
- Auto-correct and predictive typing
- Language translation tools
- Social media sentiment tracking
- Document search and recommendation systems

Implementation : Sentiment Analysis

```
import pandas as pd
import numpy as np
import re
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from nltk.tokenize import word_tokenize
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, classification_report
# 1. DOWNLOAD NECESSARY NLP TOOLS
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('omw-1.4')
```

```
import os
from google.colab import files
# 1. Upload kaggle.json
if not os.path.exists('kaggle.json'):
    print("Please upload your kaggle.json file:")
    uploaded = files.upload()
# 2. Setup Kaggle Environment
!mkdir -p ~/.kaggle
!cp kaggle.json ~/.kaggle/
!chmod 600 ~/.kaggle/kaggle.json
# 3. Download and Unzip Dataset
!kaggle datasets download -d kazanova/sentiment140
!unzip sentiment140.zip
# 4. Load the file
columns = ['target', 'ids', 'date', 'flag', 'user', 'text']
df = pd.read_csv('training.1600000.processed.noemoticon.csv', encoding='latin-1', names=columns)
# 3. BALANCED SAMPLING (50,000 rows for speed and memory)
df_neg = df[df['target'] == 0].sample(25000, random_state=42)
df_pos = df[df['target'] == 4].sample(25000, random_state=42)
df_final = pd.concat([df_neg, df_pos]).sample(frac=1).reset_index(drop=True)
df_final['target'] = df_final['target'].replace(4, 1) # Convert 4 to 1 for Positive
# 4. ADVANCED DATA CLEANING FUNCTION
lemmatizer = WordNetLemmatizer()
stop_words = set(stopwords.words('english'))
def clean_text(text):
    text = str(text).lower()
    text = re.sub(r'@[A-Za-z0-9]+', '', text)       # Remove @mentions
    text = re.sub(r'https?://\S+', '', text)        # Remove URLs
    text = re.sub(r'[^a-zA-Z ]', '', text)          # Remove special characters
```

```
    tokens = word_tokenize(text)
    # Lemmatization and basic Stop Word removal
    cleaned = [lemmatizer.lemmatize(w) for w in tokens if w not in stop_words]
    return " ".join(cleaned)
print("Cleaning data... please wait.")
df_final['processed_text'] = df_final['text'].apply(clean_text)
# 5. FEATURE EXTRACTION (TF-IDF with Bigrams)
vectorizer = TfidfVectorizer(ngram_range=(1, 2), max_features=50000)
X = vectorizer.fit_transform(df_final['processed_text'])
y = df_final['target']
# 6. TRAIN-TEST SPLIT & MODEL TRAINING
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
model = MultinomialNB(alpha=0.1) # Optimized Alpha for better sensitivity
model.fit(X_train, y_train)
# 7. EVALUATION
y_pred = model.predict(X_test)
print(f"\nFinal Accuracy: {accuracy_score(y_test, y_pred) * 100:.2f}%")
# 8. LIVE PREDICTION FUNCTION
def predict_sentiment():
    print("\n" + "="*40)
    user_input = input("Mam, please enter a sentence to test: ")
    cleaned = clean_text(user_input)
    vectorized = vectorizer.transform([cleaned])
    prediction = model.predict(vectorized)[0]
    # Calculate probabilities for the Viva explanationnn
    probs = model.predict_proba(vectorized)[0]
    result = "POSITIVE 😊" if prediction == 1 else "NEGATIVE ☹️"
    print(f"\nResults for: '{user_input}'")
    print(f"Confidence: {max(probs)*100:.2f}%")
    print(f"Predicted Sentiment: {result}")
```

```
    print("="*40)
```

Run the test

```
print("Name: Jyoti Verma     URN:2302573")
predict_sentiment()
predict_sentiment()
predict_sentiment()
```

```
Name: Jyoti Verma     URN:2302573

========================================
Mam, please enter a sentence to test: this movie is good.

Results for: 'this movie is good.'
Confidence: 82.74%
Predicted Sentiment: POSITIVE 😊
========================================

========================================
Mam, please enter a sentence to test: This is the worst day of my life

Results for: 'This is the worst day of my life'
Confidence: 75.97%
Predicted Sentiment: NEGATIVE 😳
========================================

========================================
Mam, please enter a sentence to test: you are very intelligent

Results for: 'you are very intelligent'
Confidence: 75.22%
Predicted Sentiment: POSITIVE 😊
```